QuillAudits

AUDIT REPORT

May 2026

For

atomyx

Table of Content

Executive Summary

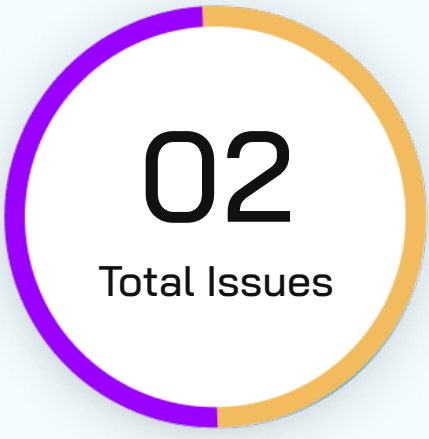
Project Name	Atomyx
Project URL	https://atomyx.capital/
Overview	Atomyx Evm tokencontract is a whitelist-restricted ERC20 token built using OpenZeppelin's ERC20 and Ownable standards. It allows only whitelisted addresses to send and receive tokens, enforcing transfer restrictions through custom validation logic. The owner has administrative control to add or remove single or multiple addresses from the whitelist. The contract also supports owner-controlled token minting and burning functionalities. Custom error handling and restriction messages improve transaction transparency and debugging. Ownership renunciation is permanently disabled to ensure continuous administrative control over whitelist management and token operations. Additionally, the contract supports configurable token decimals and stores extra metadata information during deployment.
Audit Scope	The scope of this Audit was to analyze the atomyx Smart Contracts for quality, security, and correctness.
Source Code link	https://gitlab.com/ideadlt/eternyze/public-token-manager/-/blob/master/src/contracts/evm/token-contract.sol?ref_type=heads
Contracts in Scope	src/contracts/evm/token-contract.sol
Branch	Master
Commit Hash	acff38b2616574c4215f016a21f03be27311c9f2
Language	Solidity
Blockchain	Polygon
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	4th May 2026 - 7th May 2026
Updated Code Received	14th May 2026
Review 2	14th May 2026

Fixed In	fa3b0536ca5d3c506379f6873ca56bf9054aa381

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	0 (0.0%)
Low	1 (50.0%)
Informational	1 (50.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	**1**
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	**1**	0

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Missing whitelist validation in mint() allows minting to non-whitelisted addresses	Low	Resolved
2	Centralization risk owner compromise can fully control token policy and balances	Informational	Acknowledged

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Low Severity Issues

Missing whitelist validation in mint() allows minting to non-whitelisted addresses

Resolved

Path

token-contract.sol

Function Name

`mint()`

Description

The mint(address account, uint256 amount) function is restricted to onlyOwner, but it does not verify whether account is whitelisted before calling _mint(account, amount).
This is inconsistent with the contract's transfer controls, where both transfer() and transferFrom() enforce whitelist checks on participants. As a result, the owner can mint tokens to addresses that are not allowed by the whitelist policy.

Recommendation

Enforce whitelist validation inside mint() before minting

Informational Issues

Centralization risk owner compromise can fully control token policy and balances

Acknowledged

Path

token-contract.sol

Function Name

`mint()`,`burnByOwner()`...

Description

The contract uses a single privileged owner model with powerful administrative capabilities. The owner can mint new tokens (mint), burn tokens from any account (burnByOwner), and arbitrarily add or remove addresses from the whitelist (addToWhitelist, addBatchToWhitelist, removeFromWhitelist). If the owner private key is compromised, an attacker can take full control over supply and transfer permissions.

This creates a centralization risk and a single point of failure. A compromised owner can inflate supply, censor users by removing them from whitelist, force-burn user balances, and redirect economic value.

Recommendation

Reduce privileged-key risk and add layered controls :
- Replace single EOA ownership with a multisig wallet for owner operations.
- Add a timelock for sensitive admin actions (mint, burnByOwner, addToWhitelist..).

Functional Tests

Some of the tests performed are mentioned below:

- deploy sets owner to deployer (Ownable(msg.sender))
- deploy sets _decimals to tokenDecimals input
- deploy mints initialSupply to deployer balance
- deploy sets whitelist[deployer] = true
- deploy sets whitelist[omniBusWalletAddress] = true
- deploy sets metadata to _metadata input
- decimals() returns constructor tokenDecimals value
- decimals() remains constant across transfers/mints/burns
- returns SUCCESS when from and to are both whitelisted
- returns SENDER_NOT_ALLOWED when from is not whitelisted
- returns RECEIVER_NOT_ALLOWED when from is whitelisted and to is not whitelisted
- returns “SUCCESS” for code SUCCESS
- returns “Sender not in whitelist” for code SENDER_NOT_ALLOWED
- returns “Receiver not in whitelist” for code RECEIVER_NOT_ALLOWED
- transfer succeeds when sender and receiver are whitelisted
- transfer reverts with “Sender not in whitelist” when sender is non-whitelisted
- transfer reverts with “Receiver not in whitelist” when receiver is non-whitelisted
- transfer keeps ERC20 accounting correctness on successful path
- transferFrom succeeds when from/to are whitelisted and allowance is sufficient
- transferFrom reverts with “Sender not in whitelist” when from is non-whitelisted
- transferFrom reverts with “Receiver not in whitelist” when to is non-whitelisted
- transferFrom reverts with OZ allowance error when allowance is insufficient
- transferFrom decreases allowance on success per ERC20 behavior
- only owner can call onlyOwner functions (non-owner call reverts)
- owner can mint and increase totalSupply and receiver balance

- burns tokens from target account and decreases totalSupply
- reverts if account balance is insufficient for burn amount
- owner can burn from arbitrary user account (centralization/trust risk)
- ownership remains unchanged after attempted renounce
- current implementation allows minting to non-whitelisted account

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

Contract Name	Function	Category
TokenContract	constructor(string name, string symbol, uint256 initialSupply, uint8 tokenDecimals, address omniBusWalletAddress, string _metadata)	**What this function can do** 1. Initialize token metadata and decimals. 2. Mint initialSupply to deployer. 3. Whitelist deployer and omnibus wallet. 4. Set on-chain metadata string. **What this function cannot/should not do** 1. Cannot be called again after deployment. 2. Should not accept invalid operational addresses .
TokenContract	decimals()	**What this function can do** 1. Return configured token decimals. **What this function cannot/should not do** 1. Must not modify state
TokenContract	detectTransferRestriction(address from, address to)	**What this function can do** 1. Evaluate whitelist restriction result for sender/receiver. 2. Return SUCCESS, SENDER_NOT_ALLOWED, or RECEIVER_NOT_ALLOWED. **What this function cannot/should not do** 1. Must not move tokens or change state. 2. Should not be treated as full policy enforcement for mint/burn paths.
TokenContract	messageForTransferRestriction(uint8 restrictionCode)	**What this function can do** 1. Convert restriction codes into human-readable messages. **What this function cannot/should not do** 1. Must not modify state. 2. Should not return misleading strings for known codes.

Contract Name	Function	Category
TokenContract	transfer(address to, uint256 amount)	**What this function can do** 1.Transfer tokens only when sender and receiver pass whitelist checks. **What this function cannot/should not do** 1. Must not bypass whitelist policy. 2. Must not succeed when restriction code is non-success.
TokenContract	transferFrom(address from, address to, uint256 amount)	**What this function can do** 1. Move tokens on allowance path if whitelist checks pass. **What this function cannot/should not do** 1. Must not bypass whitelist policy. 2. Must not bypass allowance constraints.
TokenContract	addToWhitelist(address account)	**What this function can do** 1. Add single account to whitelist. 2. Emit WhitelistUpdated when state changes. **What this function cannot/should not do** 1. Non-owner must not call successfully. 2. Should not emit duplicate event if already whitelisted.
TokenContract	removeFromWhitelist(address account)	**What this function can do** 1. Remove single account from whitelist. 2. Emit WhitelistUpdated when state changes. **What this function cannot/should not do** 1. Non-owner must not call successfully. 2. Should not emit duplicate event if already removed.
TokenContract	addBatchToWhitelist(address[] accounts)	**What this function can do** 1. Add multiple accounts to whitelist in loop. Emit per-account event for newly added entries only. **What this function cannot/should not do** 1. Non-owner must not call successfully. 2. Should not alter already-whitelisted entries.

Contract Name	Function	Category
TokenContract	mint(address account, uint256 amount)	**What this function can do** 1. Mint new tokens to target account by owner. **What this function cannot/should not do** 1. Non-owner must not call successfully.
TokenContract	burnByOwner(address account, uint256 amount)	**What this function can do** 1. Burn tokens from any target account by owner. **What this function cannot/should not do** 1. Non-owner must not call successfully..
TokenContract	renounceOwnership()	**What this function can do** 1. Explicitly block ownership renunciation by reverting. **What this function cannot/should not do** 1. Must never succeed.

Closing Summary

In this report, we have considered the security of Atomyx Token. We performed our audit according to the procedure described above.

No critical issues in Atomyx token, just 2 issues one Low and one Informational severity were found during Audit.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

May 2026

For

atomyx

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com